

Research Report: POCKET Subagent Mesh Protocol & CloudColony Wrap

Date: 2026-07-28

Subject: Formalization of Antigravity-style mesh coordination, always-use hook, and framework packaging

Lead: ItsNotAI Labs / Medina Tech Labs

Classification: Strategic Engineering / Beta Protocol

Archival Code: `INL-2026-POCKET.MESH.v1`

Protocol ID: `MEDINA-SUBAGENT-MESH/1.0`

Runtime: `pocket.agent\_hook` · `pocket.protocols.subagent\_mesh\_protocol` · `pocket.mesh\_disk`

API: `GET /v1/protocols/mesh` · `POST /v1/hooks/mesh` · `POST /v1/subagents/dispatch`

1. Executive Summary

This report formalizes how POCKET coordinates **many** subagents without coupling them to a single chat transcript. The design is inspired by Antigravity-style worker meshes: agents hold **SHA identities**, exchange **HMAC-signed** (and body-ciphered) envelopes, leave **artifacts** on a **virtual mesh disk** (prefer `E:\POCKET\_MESH`), and tune **frequency channels** (`freq-0` ... `freq-5`).

The **always-use doctrine** is enforced by `pocket.agent\_hook.ensure\_mesh\_hook()`, which arms bootstrap + design agents + four headless workers whenever the host serves. CloudColony remains a **separate framework wrap** (`E:\repos\cloudcolony-sovereign`) for Triple Protocol / MCP colonies / ICP — product builds (POCKET, full MESIE) consume it; they are not the framework itself.

Claim	Status
Virtual mesh root on E:	Live — `E:\POCKET_MESH`
SHA agent IDs + HMAC envelopes	Live — `mesh_disk`
Body cipher (`hmac-sha256-xor-v1`)	Live — encrypt on send, decrypt on inbox read
Frequency channels	Live — `freq-0`...`freq-5` lanes
12 Latin + 4 design + 4 headless	Live — ≥20 registered core
`@dispatch` UI + API	Live — `/v1/subagents/dispatch`
Always-use hook on `serve()`	Live — `agent_hook`
Microsoft + Bluetooth/HZ protocols	Live — thin host bridges
CloudColony standalone package	Live — `cloudcolony` on E:\repos
Full BLE radio transport	Planned — file-bus default
AES/NaCl sealed per-agent keys	Planned — keys/ dirs reserved

2. Problem Statement

2.1 Chat-coupled agents

Typical multi-agent demos share one transcript. That fails for:

- Long-running headless work (forge, ship checklist)
- Cross-agent handoffs without human-visible noise
- Offloading I/O onto high-capacity storage (5TB E:)
- Integrity / attribution of who wrote what

2.2 Required properties

1. **Identity** — stable, host-salted SHA IDs
2. **Integrity** — HMAC over message cores
3. **Confidentiality (local)** — body cipher on bus
4. **Transport** — frequency channels + mailboxes
5. **Persistence** — artifacts + virtual disk workspaces
6. **Always-on** — protocol armed without manual bootstrap

3. Architecture

3.1 Logical stack

```

    ...
[Composer / API / Orchestrator]
■ @mentions · route_message · dispatch_named
▼
[agent_hook] ■■ always-use arm ■■■ bootstrap + headless + design
■
▼
[subagent_dispatch] ■■ execute Latin/design/headless skills
■
▼
[mesh_disk] ■■ SHA · HMAC · body_cipher · vdisk · workers
■
■■■ agents/<ID>/{inbox,outbox,artifacts,keys}
■■■ channels/freq-N.jsonl
■■■ vdisk/workspaces/<ID>/
■■■ workers/worker_*.py (E: offload)
■
▼
[protocols] microsoft · bluetooth_hz · subagent_mesh_protocol
    ...

```

3.2 Virtual disk (not OS VHD)

We intentionally use a **logical virtual workspace** under `E:\POCKET\_MESH\vdisk` rather than mounting a `.vhd`. Rationale:

- Zero admin privilege for beta
- Same organizational benefit (agents write off C:)
- Portable via `POCKET\_MESH\_ROOT`
- Workers under `E:\POCKET\_MESH\workers` can run independently of the chat process

3.3 Encryption model

Layer Mechanism
----- -----

```

| Identity | `SHA-256(salt \|\| agent_id)` in `id.json` |
| Envelope auth | `HMAC-SHA256` over sorted JSON core → `hmac_sha256` |
| Body privacy | `body_cipher`: nonce + XOR keystream from HMAC blocks (`hmac-sha256-xor-v1`) |
| Channel log | Stores ciphertext when encrypt=True; plaintext field becomes `[encrypted]` |
| Inbox read | Verifies HMAC; fills `body_plain` from cipher |

```

****Research note:**** This is ****host-local privacy + integrity****, not end-to-end multi-tenant crypto. Future work: per-agent keypairs under `keys/`, AES-GCM, cross-host mesh.

4. Agent Packs

4.1 Latin core (12)

```

| ID | Role class |
|----|-----|
| ARCHON | Multimodal desk alpha |
| HYDRA | Parallel fan-out |
| SCRUTATOR | Research / inspect |
| SCRIPTOR | Compose |
| PORTARIUS | Open apps |
| OCULUS | Sense / screenshot |
| SPECULUM | Screen record |
| REPOSITOR | Repos / git |
| CONSILIARIUS | Copilot |
| TABELLARIUS | Outlook |
| NAVIGATOR | Browser multi-step |
| GUPPY | Commercial multi-step |

```

4.2 Design pack (4) — first-class

```

| ID | Focus |
|----|-----|
| DESIGN | Product craft / cohesion |
| AESTHETE | Visual taste |
| LAYOUT | Structure / spacing |
| MOTION | Feedback / transitions |

```

****Doctrine:**** `@DESIGN` maps to ****DESIGN****, never SCRIPTOR.

4.3 Headless pack (4) — powerful background

```

| ID | Duty | E: worker |
|----|-----|-----|
| FORGE_HEADLESS | Build / test / package | `worker_forge.py` |
| SENTINEL_HEADLESS | Security / sanity | `worker_sentinel.py` |
| RESEARCH_HEADLESS | Research + RFE notes | `worker_research.py` |
| SHIP_HEADLESS | Beta / release checklist | `worker_ship.py` |

```

Heartbeats publish on ****freq-1**** every ~120s when pack is started.

5. MEDINA-SUBAGENT-MESH/1.0 — Always-Use Doctrine

Formalized in `pocket.protocols.subagent\_mesh\_protocol`.

5.1 Doctrine flags

```

| Flag | Default |
|-----|-----|
| `always_use_mesh` | true |
| `always_bootstrap_on_serve` | true |
| `always_start_headless` | true |

```

```
| `dispatch_on_at_mentions` | true |
| `artifacts_without_shared_chat` | true |
| `prefer_virtual_disk_e` | true |
| `design_first_class` | true |
| `encrypt_bodies_default` | true |
```

5.2 Always-use hook

Module: `pocket.agent\_hook`

```
| Function | Behavior |
|-----|-----|
| `ensure_mesh_hook()` | Idempotent arm: bootstrap + design + headless |
| `route_message(text)` | Parse `@` → `dispatch` via mesh |
| `dispatch_named(agent, msg)` | Explicit target without chat |
| `protocol_report()` | Research/ops snapshot |
```

****Host integration:**** `server.serve()` calls `ensure\_mesh\_hook()` before accepting traffic. Env:

```
| Env | Effect |
|-----|-----|
| `POCKET_MESH_HOOK=0` | Disable arming |
| `POCKET_ALWAYS_MESH=0` | Same (master off) |
| `POCKET_HEADLESS_AUTO=0` | Bootstrap only, no headless threads |
| `POCKET_MESH_ROOT=path` | Override mesh root |
```

5.3 Frequency map

```
| Lane | Channel | Purpose |
|-----|-----|-----|
| user | freq-0 | User @dispatch |
| heartbeat | freq-1 | Headless pulse |
| design | freq-2 | Design bus |
| security | freq-3 | Sentinel |
| ship | freq-4 | Release |
| intel | freq-5 | Research / BLE stubs |
```

6. Related Protocols

6.1 Microsoft system protocol

`pocket.protocols.microsoft\_protocol` – thin safe hooks: UIA click, maximize, allow-listed open, page render, guarded shell. No raw kernel drivers.

6.2 Bluetooth / Hz

`pocket.protocols.bluetooth\_hz` + `pocket.hz\_mesh` – map BLE-style MHz bands to `freq-N`. Physical BLE is optional; ****file-bus** is the default real-time transport

6.3 CloudColony Triple Protocol (framework wrap)

```
| ID | Role |
|-----|-----|
| MEDINA-LOOM/0.1 | Vault + council + signal |
| MEDINA-MCP-COLONY-DEPLOY/1.0 | MCP colony servers |
| MEDINA-COLONY-BRIDGE/1.0 | Airgap + Capsula + ICP |
```

****POCKET mesh**** is host-local agent coordination. ****CloudColony**** is the sovereign colony framework. They compose; they are not the same package.

7. CloudColony Wrap vs Product Build

Layer	Repo / path	Role
Framework wrap	`E:\repos\cloudcolony-sovereign`	Public `cloudcolony` package, MCP colonies, ICP, Triple Protocol
Host product	`OneDrive\pocket-os` (POCKET)	Desktop + API + Studio, Fusion Sense, mesh hook
Full monorepo	MESIE	Spectral engines, datasets, product extras

Doctrine for funding/viral demos: ship **frameworks + wrappers + agents** so benchmarks and videos target a stable API, not an opaque app binary.

Canonical standalone: `E:\repos\cloudcolony-sovereign`

Source of truth for deep modules: MESIE monorepo `mesie/cloud` + export script.

8. API Surface

Method	Path	Purpose
GET	`/v1/protocols/mesh`	Protocol + hook report
POST	`/v1/hooks/mesh`	Force arm hook
GET	`/v1/subagents`	Unified roster + mesh meta
POST	`/v1/subagents/dispatch`	@dispatch / named agents
GET	`/v1/mesh`	Mesh status (auto-hook)
GET	`/v1/mesh/inbox/{agent}`	Signed inbox
GET	`/v1/mesh/channel?name=freq-0`	Channel tail
POST	`/v1/mesh/send`	Peer message
POST	`/v1/mesh/bootstrap`	Force bootstrap + headless
POST	`/v1/headless/start` \ `stop`	Headless pack control

UI: composer `@` autocomplete; dispatch chips (no raw JSON dumps); rail shows mesh count + **E: mesh** pill.

9. Empirical Status (2026-07-28)

Measured on this host:

- Mesh root: `E:\POCKET\_MESH`
- Agent homes: **≥21** (core packs + USER)
- Headless: **4** identities + heartbeat artifacts
- Design: DESIGN/AESTHETE/LAYOUT/MOTION leave critique + CSS on mesh and vdisk
- Channel traffic: `freq-0`, `freq-1`, `freq-2` observed
- Hook arm: `ensure\_mesh\_hook()` returns `armed=True` with empty errors on smoke
- CloudColony: `import cloudcolony` version **0.1.0**, smoke import OK

10. Viral / Funding Framing

To make the platform legible externally:

- Research papers** (this doc + RFE + Fusion Sense) — technical, not landing fluff
- Framework repo** (CloudColony) — demos, benchmarks, `pip install -e`
- Mesh protocol ID** — cite `MEDINA-SUBAGENT-MESH/1.0` in videos
- Always-use hook** — demo “agents talk without chat” via artifacts on E:
- Product channels** — Desktop / API / Studio as packaging, not competing brands

11. Open Research Questions

1. Per-agent sealed keys vs host salt — multi-user mesh?
2. BLE physical transport mapping to SHA IDs — Hz mesh productization
3. Cross-host mesh federation via CloudColony bridge
4. Headless FORGE depth — real package builds vs smoke workers
5. Formal verification of HMAC envelopes under concurrent writers
6. Desk “ten” curated roster vs full 20 for UI density

12. Module Reference

Module	Path
Protocol constants	`pocket/protocols/subagent_mesh_protocol.py`
Always-use hook	`pocket/agent_hook.py`
Virtual disk + crypto	`pocket/mesh_disk.py`
@dispatch	`pocket/subagent_dispatch.py`
Design pack	`pocket/design_agents.py`
Panel roster	`pocket/subagents_panel.py`
Hz lanes	`pocket/hz_mesh.py` · `protocols/bluetooth_hz.py`
Microsoft	`protocols/microsoft_protocol.py`
Ops doc	`docs/SUBAGENT_MESH.md`
CloudColony	`E:\repos\cloudcolony-sovereign`

13. Conclusion

****MEDINA-SUBAGENT-MESH/1.0**** makes “use subagents this way” an enforceable host doctrine: virtual disk on E:, signed/ciphered mail, frequency channels, design + headless packs, and an always-on hook at serve time. CloudColony remains the separate framework wrap for colony-scale protocols. Together they form the research and product story for beta: ****frameworks, wrappers, and agents that coordinate without living inside one chat.****

— End of report · `INL-2026-POCKET.MESH.v1`